Sabor & Clic ? Relatório de Alterações e Funcionamento

Documento gerado após a auditoria e correção do sistema. Resume (1) todas as alterações feitas, (2) os erros corrigidos sem solicitação explícita, mas necessários, e (3) um resumo do funcionamento do código.

Observação importante sobre verificação: o ambiente de execução não possui acesso à internet para instalar as dependências (Flask, SQLAlchemy, etc.), por isso NÃO foi possível executar a aplicação web completa (boot + navegação). A verificação realizada foi: (a) compilação de todos os módulos Python; (b) validação estrutural dos dados de seed; (c) testes do esquema e da trigger de baixa de estoque em SQLite puro (biblioteca padrão), inclusive com os dados reais de seed. A lógica central de estoque foi validada ponta a ponta dessa forma.

---

1. Bloqueadores que impediam a aplicação de iniciar (corrigidos)

1. seeds.py ? vírgulas ausentes entre tuplas em lista_pratos, em todas as listas ingredientes_* e em ficha_bebidas. Sem as vírgulas, Python interpretava "(tupla)(tupla)" como chamada de função e lançava TypeError no import. Como database.py faz "from seeds import *", a aplicação inteira não subia. Correção: inseridas as vírgulas finais ausentes (153 linhas) e corrigidas 5 tuplas de lista_pratos que tinham comentário após o parêntese.

2. controller/admin/__init__.py ? import incorreto "from admin import ..." e nomes de módulo inexistentes (admin_lotes, admin_logatividade). Os arquivos reais são admin_lote.py e admin_logs.py. Isso gerava ImportError e o blueprint admin não era registrado. Correção: imports passados para o caminho de pacote "controller.admin" e nomes de módulo corrigidos. Todos os submódulos admin passaram a importar "from controller.admin import ...".

3. seeds.py ? construtores de modelos chamados com argumentos posicionais (Ingrediente(...), Lote(...), FichaTecnica(...), FichaTecnica_bebida(...), ServicoBancada(...), ServicoCardapio(...), PratoCardapio(...)). Modelos SQLAlchemy só aceitam argumentos nomeados. Correção: todas as construções passaram a usar argumentos nomeados.

4. seeds.py ? outros erros de carga: campo "email_criptogradafo" (grafia errada de email_criptografado); uso de ".id" em vez de ".id_ingrediente"; iniciar_prato_cardapio usava buscar_servico_por_nome para buscar pratos, desempacotava 2 valores de tuplas de 4 e omitia o campo obrigatório qtd. Nomes com espaços nas pontas (ex.: "Bobo_de_camarao ") não casavam nas associações. Correção: grafia e atributo corrigidos; iniciar_prato_cardapio reescrito corretamente; normalização com strip aplicada na criação e na busca por nome em todas as funções iniciar_*.

---

2. Estoque: baixa automática, concorrência e segurança

5. Trigger de baixa de estoque: a lógica SQL (triggers.py) estava correta (consolida pratos + bebidas, valida estoque, distribui a baixa entre lotes por validade, abate lote e ingrediente). O problema era o DISPARO: a trigger era

(re)criada dentro de criar_pedido e DEPOIS do commit em aceitar_pedido. Correção: a trigger passou a ser criada uma única vez em database.init_app (após db.create_all), de forma idempotente. Removida a recriação frágil dentro de criar_pedido.

6. Concorrência (dois chefs aceitando o mesmo pedido): não havia controle. Correção: criada a função aceitar_pedido_atomico (pedido_model) que faz um UPDATE condicional "WHERE id_chefe IS NULL AND status='aguardando'". Apenas um chef consegue afetar a linha; o segundo falha silenciosamente. Como a baixa só ocorre na transição para 'confirmado', não há baixa dupla.

7. Baixa dupla: a condição da trigger (OLD.status <> 'confirmado') já impede re-disparo no caminho normal. Além disso, o aceite atômico impede reconfirmar um pedido já confirmado. Verificado em teste: reconfirmar não baixa novamente.

8. Permissão do chef (IDOR): qualquer chef podia alterar qualquer pedido. Correção: criada atualizar_status_pelo_chefe, que só permite alteração quando pedido.id_chefe == chef logado; valida também o conjunto de status válidos.

9. Alerta de estoque mínimo ao administrador: não existia. Correção: criada listar_ingredientes_abaixo_minimo (ficha_model) e exibida como alerta de reposição na página de estoque do admin (qtd <= qtd_min).

10. Pedido sem estoque / convidados <= 0: a trigger aborta quando falta estoque (RAISE ABORT), e o aceite atômico captura o erro e falha com mensagem. O número de convidados é validado (> 0) em fazer_pedido e também pela CheckConstraint do banco.

---

3. Relatórios do administrador (corrigidos)

11. pedidos_status sempre retornava tudo zerado: listar_pedidos_periodo gerava "status IS NULL" quando status=None. Correção: o filtro de status só é aplicado quando status is not None (em listar_pedidos_periodo e listar_pedidos_entre_datas).

12. ticket_medio_periodo tinha uma linha duplicada que reintroduzia um ZeroDivisionError (sobrescrevia o cálculo protegido). Correção: linha removida; todos os caminhos de divisão estão protegidos.

13. CardapioPedido não gravava id_servico (parte da chave primária), o que quebrava a integridade e o relatório pedidos_tipo. Correção: criar_pedido e criar_cardapio_pedido passaram a gravar id_servico.

---

4. Fluxo do cliente (carrinho, pedido, bebida, perfil, endereço)

14. fazer_pedido estava quebrado: ordem de argumentos de criar_pedido trocada, total somava convidados em vez de preço, f-string com aspas triplas aninhadas (erro de sintaxe que quebrava o módulo inteiro), ausência de return. Correção: função reescrita ? valida endereço (e verifica posse), data/horas, convidados (> 0); calcula o total como soma(preço_base x convidados) + bebidas; cria o

pedido; limpa o carrinho; envia e-mail de confirmação.

15. PedidoBebida recebia "valor_unitario" (coluna inexistente; a coluna é valor_total). Correção: passou a gravar valor_total = valor_unitario x qtd em criar_pedido e adicionar_bebida_ao_pedido.

16. cancelar_pedido não recebia id por URL e não checava posse (IDOR). Correção: rota passou a ser /cancelar_pedido/<int:id_pedido> e valida pedido.id_cliente == cliente logado.

17. dados_perfil usava session("id") (chamada inválida), quebrava quando não havia pedido ativo e tratava associações como se fossem cardápios. Correção: usa session.get("id"), trata ausência de pedido, resolve Cardapio/Servico a partir de CardapioPedido e adiciona a etapa de acompanhamento (Recebido/Em preparo/Entregue).

18. Mapeamento de status para o cliente: criada a função _etapa_cliente (aguardando/confirmado -> Recebido; em preparo -> Em preparo; concluido -> Entregue; cancelado -> Cancelado).

19. adicionar (carrinho): validação de convidados agora é segura contra None; o redirecionamento corrigido (slug no url_for, não no redirect); subtotal calculado (preço_base x convidados).

20. adicionar_endereco convertia numero com int() (quebra em "123A"); a coluna é String. Correção: numero gravado como string.

21. buscar_bebidas_pedido não existia (era importada). Correção: função criada em pedido_model.

---

5. Modelos e relacionamentos

22. bebidaCardapio tinha FK errada: ForeignKey("bebida.id_servico"). Correção: ForeignKey("bebida.id_bebida").

23. Endereco estava declarada DUAS vezes em usuario_model. Correção: removida a duplicata (evita conflito de tabela no SQLAlchemy).

24. criar_lote/atualizar_lote referenciavam id_bebida (coluna inexistente no modelo Lote). Correção: id_bebida removido; lote vinculado a ingrediente.

25. registrar_log tem assinatura (acao, descricao), mas várias funções de servico_model e ficha_model passavam id_usuario como primeiro argumento e exigiam id_usuario obrigatório. Correção: id_usuario tornou-se opcional e as chamadas a registrar_log foram ajustadas para (acao, descricao) ? o usuário já é obtido da sessão.

26. para_data(dia, mes, ano) produzia datas erradas (trocava ano e dia). Correção: assinatura padronizada para (ano, mes, dia), coerente com as chamadas e com a docstring.

27. listar_cardapios_do_servico retornava linhas de associação em vez de

Cardapio. Correção: passou a retornar objetos Cardapio (join), compatível com a página de serviço.

28. associar_servico_cardapio tinha default "id_usuario=int" (tipo como valor padrão) e passava "descricao" para ServicoCardapio (sem essa coluna). Correção: default None e remoção do campo inexistente.

---

6. E-mail de verificação

29. O fluxo de verificação por token (2FA no login) está correto e a expiração de 10 minutos é aplicada em buscar_token. Os envios em aceitar/atualizar pedido estavam quebrados: usavam cliente.email (atributo inexistente) e render_template com argumentos posicionais, além de pedido.id (coluna é id_pedido). Correção: chefe_controller reescrito ? e-mail enviado com o e-mail criptografado (descriptografado dentro do serviço) e render_template com kwargs (nome, id, data_confirmacao, ano). Removida a cerca markdown "```" presa no fim de email_pedido.html. config.py tornou-se tolerante a variáveis de e-mail ausentes para não derrubar a aplicação.

---

7. Páginas (HTML) criadas

Os controladores renderizavam templates inexistentes. Foram criados:

- estoque.html ? cadastro e listagem de serviços, cardápios, pratos, ingredientes, bebidas, lotes e associações (serviço-cardápio, prato-cardápio, fichas técnicas), além do alerta de estoque mínimo.
- visao_geral.html ? painel com indicadores do mês (consome o endpoint de relatórios).
- funcionarios.html e cadastrar_funcionario.html ? lista e cadastro de funcionários (chefes).
- log_atividade.html e log_atividade_detalhe.html ? trilha de auditoria.
- pedidos.html (chef) ? lista pedidos pendentes e os do chef, com botões para aceitar e atualizar status (em preparo / entregue), via JSON.
- servico.html (cliente) ? cardápios do serviço com formulário de adicionar ao carrinho.
- fazer_pedido.html (cliente) ? seleção de endereço e data/horas do evento.

Foram criadas também as rotas admin ausentes: criar serviço (/admin/servicos/criar_servico) e associar/desassociar serviço-cardápio. Os links do menu administrativo foram apontados para os endpoints reais via url_for.

---

8. Entregáveis SQL

- docs/sql/schema.sql ? DDL de todas as tabelas (a partir dos modelos) mais a view, a tabela auxiliar e a trigger de baixa de estoque (a partir de triggers.py). Validado: carrega sem erros no SQLite.
- docs/sql/seeds.sql ? INSERTs iniciais gerados a partir das listas de dados do seeds.py (cargos, serviços, bancadas, cardápios, pratos, ingredientes, bebidas,

lotes e todas as associações e fichas técnicas), com resolução de nomes para IDs. Validado: carrega sobre o schema e a trigger de baixa funciona com esses dados (testado com um pedido de Feijoada e com uma bebida à parte).

---

9. Resumo do funcionamento do sistema

Sabor & Clic é uma aplicação Flask (padrão model/controller) com SQLite via SQLAlchemy. A fábrica create_app registra quatro blueprints: auth (login, cadastro, verificação por token), cliente (home, menu, serviços, carrinho, pedido, endereços, perfil), chefe (ver/aceitar/atualizar pedidos) e admin (cadastros de estoque, funcionários, relatórios, logs). Dois hooks before_request cuidam da autenticação (exige login nas rotas privadas) e da autorização por cargo (restringe os blueprints admin e chefe).

Dados sensíveis (CPF, e-mail) são criptografados (Fernet) e indexados por HMAC para busca. A senha usa hash do Werkzeug. O login exige um segundo fator: um código de 6 dígitos enviado por e-mail, armazenado apenas como HMAC, com limite de tentativas e validade de 10 minutos.

O cliente monta o carrinho escolhendo cardápios por serviço (preço por pessoa), podendo adicionar bebidas à parte (fora do Bar). Na finalização, escolhe um endereço (próprio) e a data/horário do evento; o pedido nasce com status 'aguardando'. Os chefs veem os pedidos pendentes e podem aceitar (de forma atômica ? apenas um assume). O aceite muda o status para 'confirmado', o que dispara a trigger de baixa de estoque: para cada cardápio do pedido, somam-se os ingredientes das fichas técnicas dos pratos (quantidade por pessoa x número de convidados) e das bebidas (quantidade por unidade x quantidade pedida); valida-se a disponibilidade e, havendo estoque, abate-se dos lotes (priorizando validade mais próxima) e do estoque geral. Se faltar estoque, a confirmação é abortada.

O chef responsável atualiza o status para 'em preparo' e depois 'concluido' (entregue); o cliente acompanha essas mudanças no perfil, traduzidas para as etapas Recebido / Em preparo / Entregue. O administrador cadastra e associa serviços, cardápios, pratos, bebidas, ingredientes, lotes e fichas técnicas, consulta relatórios (quantidade de pedidos, faturamento, ticket médio, pedidos por serviço e por status, faturamento semanal), gerencia funcionários e recebe o alerta de ingredientes abaixo do estoque mínimo.